3C · AI CODING AT TEAM SCALE

How a skills library ships software.

Five diagrams of the governed workflow behind **3C** (Claude Code Coach) — the open-source framework that turns individual AI coding assistants into a team-aligned development pipeline.

01 → 05

Five diagrams. One workflow. Every role named, every gate enforced, every deviation logged.

60 sec

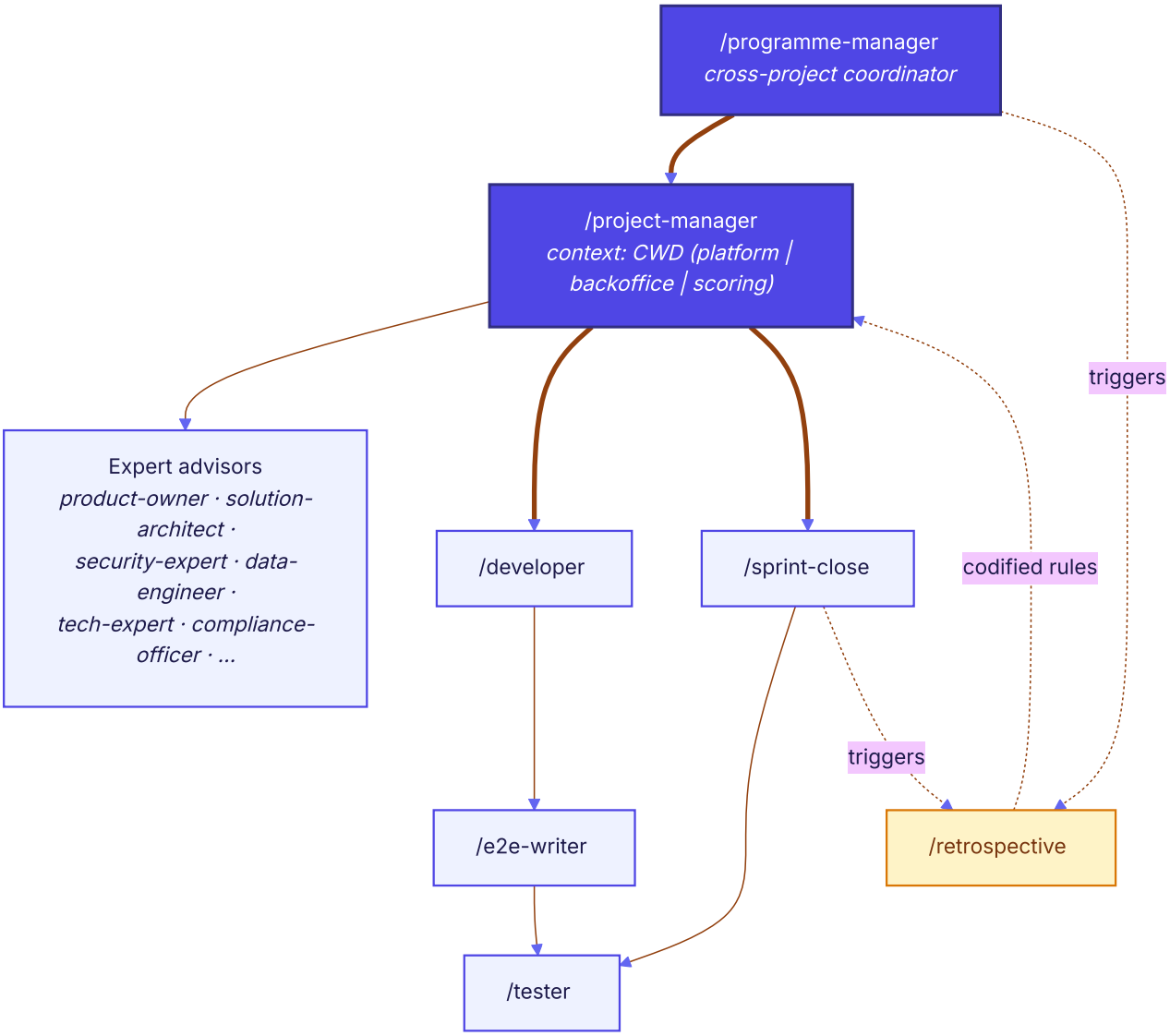
Read time. Each diagram distils months of real sprint retrospectives into a single decision or rule.

OSS

Built on the Agent Skills open standard. Portable across Claude Code, Cursor, Cline, Windsurf.

Skills talk to skills — on purpose.

The delegation chain between named roles. One `/project-manager`, context-aware across projects, orchestrating experts and the developer workflow.

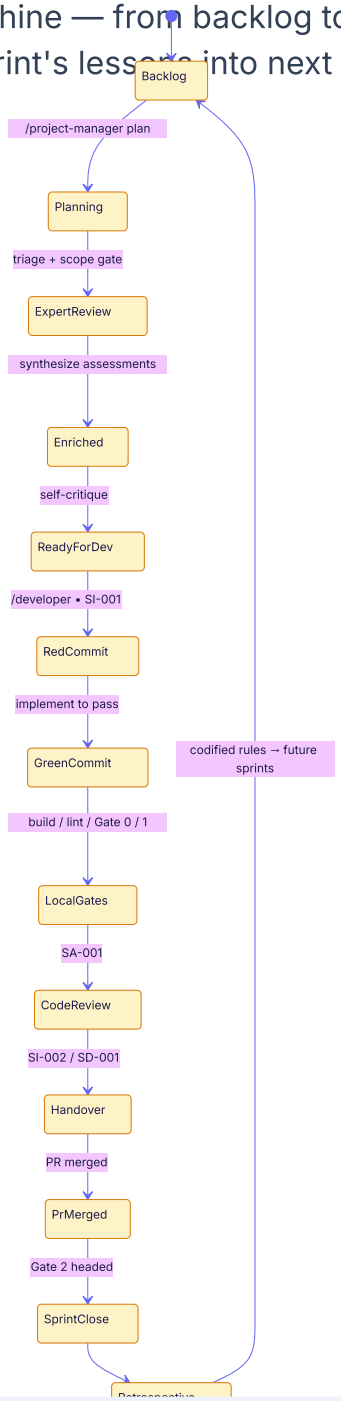


KEY MESSAGE

Skills don't work alone. A chain of named roles — from **programme-manager** down to **developer** — turns an ad-hoc AI session into a governed workflow. Every handoff is explicit, every role is accountable, and every retrospective feeds codified rules back into future sprints.

Every issue travels the same path.

The sprint lifecycle as a state machine — from backlog to retrospective, with the feedback loop that turns every sprint's lessons into next sprint's rules.

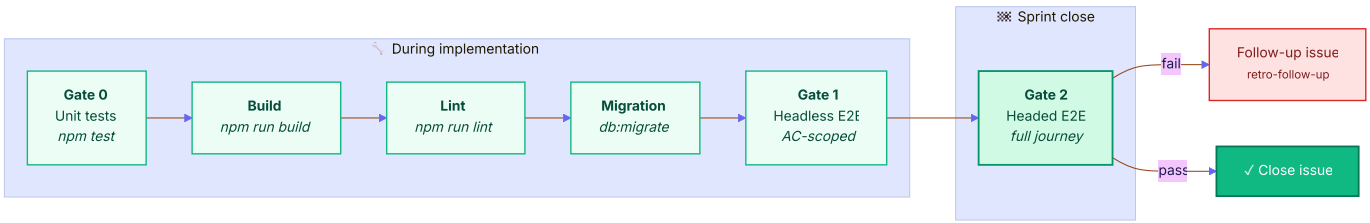


KEY MESSAGE

Fourteen states. One direction. **State-machine discipline** replaces *"just wing it"* with a repeatable, reviewable audit trail — and the **retrospective→rule→backlog** loop is the core learning mechanism that compounds team knowledge across sprints.

No issue ships past a red gate.

Six sequential gates — unit, build, lint, migration, headless E2E during implementation, and full-journey headed E2E at sprint close.

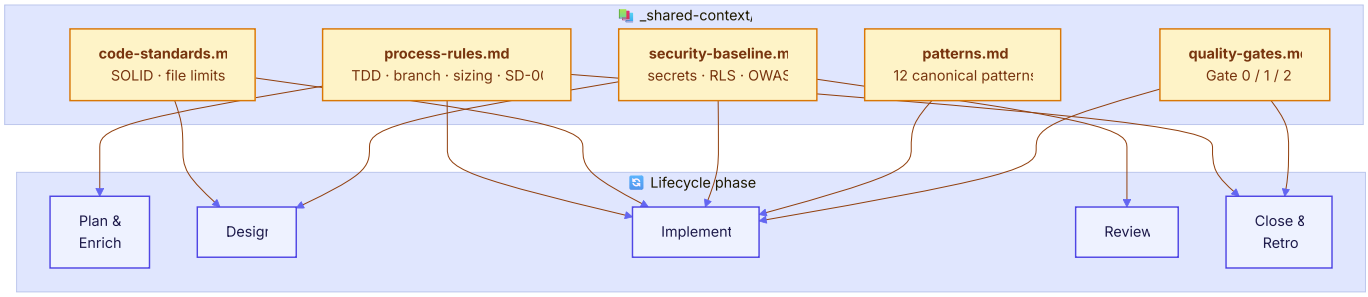


KEY MESSAGE

When a gate fails, a **follow-up issue** carries the debt forward — never a silent compromise. The gates are non-negotiable by design: a team that can't name its quality gates is a team that quietly ships the ones it doesn't run.

Context, loaded where it matters.

Five shared-context files encode the team's non-negotiables — each one injects into the specific lifecycle phases where the rules bite.

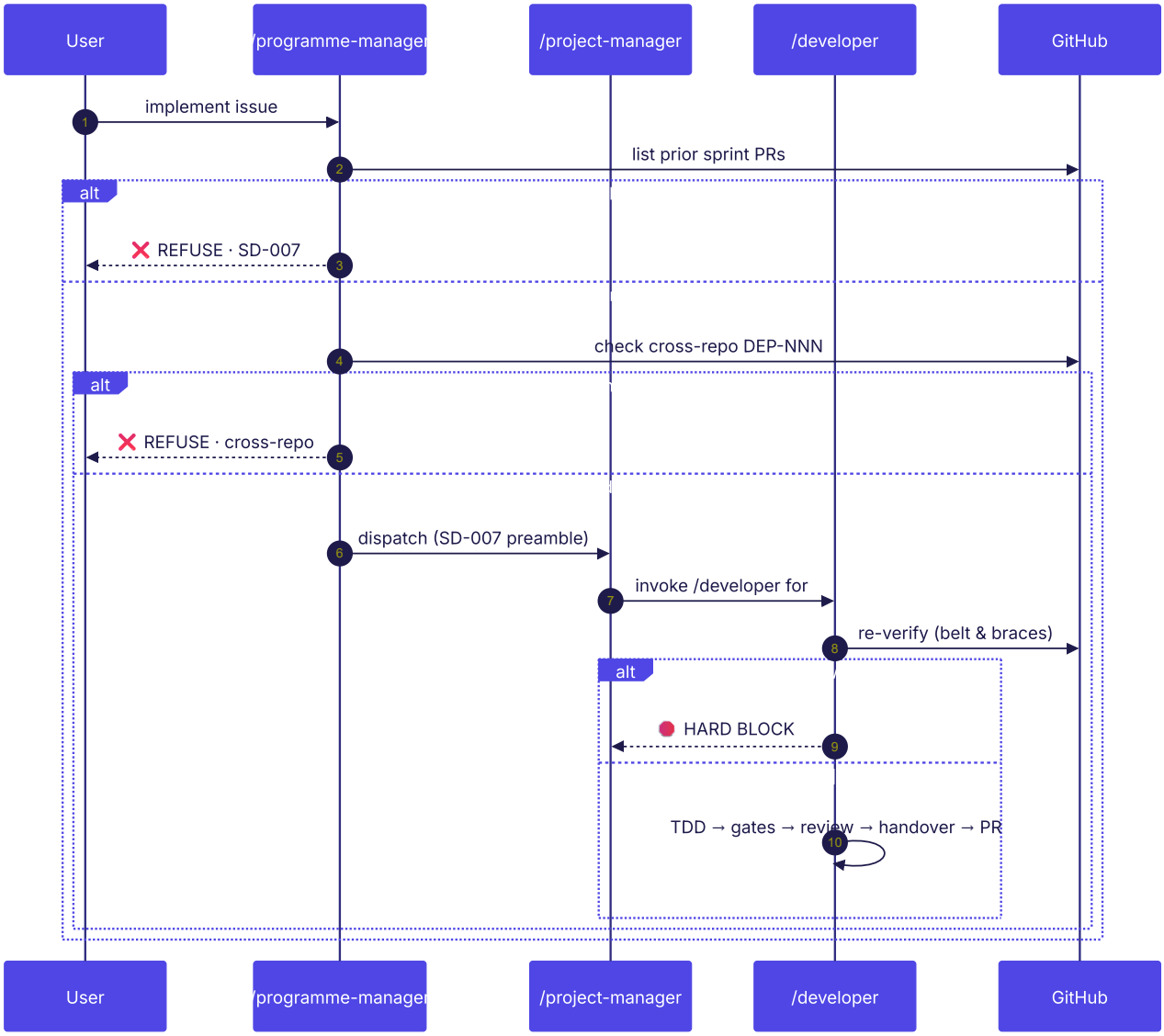


KEY MESSAGE

Progressive disclosure beats a 500-line monolith. TDD rules load during Plan and Implement. Security baseline loads during Design, Implement, and Review. Quality gates load during Implement and Close. Frontier models degrade past ~150 instructions — context has to be earned, not dumped.

Belt-and-braces, by design.

The SD-007 prior-issue merge gate. Two enforcement layers make interleaved-PR chaos structurally impossible.



KEY MESSAGE

Working on two issues in parallel feels fast — until you spend a week untangling rebase debt, duplicated contract files, and vanished commits. **SD-007 forces serialisation:** issue N's PR must merge before issue N+1 begins. Two layers enforce it — the **programme-manager** checks before dispatching, the **developer** re-checks before coding — because any single check can be bypassed by a dispatch prompt that omits it. *Cheap to check. Expensive to miss.*

THE ONE-LINE SUMMARY

Governed AI coding isn't slower. It's compounding.

Every retrospective adds a rule. Every rule loads into the right phase. Every phase has gates that can't be skipped. Every skipped gate becomes next sprint's rule.

The loop is the product. And it works the same whether you're one developer with an AI pair or a team of forty shipping to production every week.

01
Name every role. Anonymous AI work hides accountability.

02
State-machine your lifecycle. If you can't draw it, you can't audit it.

03
Gates over guidelines. Guidelines are hopes; gates are tested.

04
Load context by phase. Rules everywhere means rules nowhere.

05
Belt-and-braces enforcement. Hope is not a workflow primitive.

06
Retrospectives write rules. Rules write next sprint's diagram.

Open source. Open standard.
Borrow, fork, improve.

github.com/kgn-git/praise · built on the Agent Skills open standard

#AIEngineering
#DeveloperExperience
#ClaudeCode
#SoftwareProcess
#Governance